Essentials of Artificial Intelligence

Intake 41 – IT3182 (IT/IS)

Intake 42 – CS 22032 (DBA) / CS22023 (CS/SE/COE)

Project Proposal

Group 25



Project Details	
Project Title	An Intelligent Deepfake Image Detection System
Group Details	
Name	**Index Number**
KDST Nethmi	D/DBA/25/0039
SPKTC Perera	D/BIS/24/0007
CS Gallage	D/BIT/24/0078
SST Perera	D/BCS/25/0011

Contents

1. Abstract 3

2.Introduction and Problem Background 3

2.1 Problem statement 3

2.2 Objectives 3

3. Scope and AI Techniques 4

4. Dataset and Preprocessing 4

5. System Design 5

5.1 Intended pipeline 5

5.2 E8 architecture 6

6. Experimental Method 7

6.1 Training configuration and model summary 8

6.2 Training code 9

6.3 Example notebook outputs 10

6.4 Training time and computational cost 13

7. Threshold Selection and Model Evaluation 15

7.1 Confusion matrix 16

7.2 E8 versus E10 17

8. Implementation Status and Testing 17

9. Ethical and Social Considerations 17

10. Limitations and Future Work 18

11. Conclusion 18

12. Detailed Theory and Technical Background 18

12.1 Supervised learning and classification 18

12.3 Batch normalization and regularization 19

12.4 Model scores and decision thresholds 19

12.5 Computer vision and the three-concept requirement 19

13. Dataset Methodology and Reproducibility 20

13.1 Source, class mapping and splits 20

13.2 Data-processing steps 20
13.3 Quality controls and unresolved checks 20
14. Expanded Experiment-by-Experiment Discussion 21
14.1 Experimental progression and rationale 21
14.2 E1 - 224x224 baseline 22
14.3 E2 - augmentation 23
14.4 E3 - reduced input resolution 23
14.5 E4 and E4-Ext - higher resolution 24
14.6 E5/E6 - batch-normalization stage 25
14.7 E7 - dropout 0.3 25
14.8 E8 - deeper CNN with dropout 0.4 26
14.9 E9 - EfficientNetB0 27
14.10 E10 - additional regularization 27
15. Quantitative Evaluation and Error Analysis 28
15.1 Metric definitions 28
15.2 Detailed E8 evaluation 28
15.3 E8 versus E10 trade-off 29
15.4 Interpretation of the validation/test gap 29
15.5 Corrected experiment summary 29
15.6 Test confusion matrices 30
15.7 Class-by-class test results 30
15.8 Training and validation results 31
15.9 E8 and E10 validation results 31
15.10 Experiment hyperparameter summary 32
16. Designed Risk Assessment and Rule-Based Advice 32
17. Software Architecture and Prototype Verification 33
18. Testing Plan, Risks and Ethical Considerations 33
19. Project Management and Four-Member Participation 34

1. Abstract

Deep Guard AI researches automatic human face image classfication using convolutional neural networks (CNNs). The researchers utilized a balanced dataset of 190,335 images in all stages of the study including the training stage, validation stage and testing stage. Various experiments were conducted by comparing resolution, data augmentation, normalization, model architecture depth, transfer learning and regularization. DeepGuard AI selected the E8 batch normalized deeper CNN model for the image classfication task based on the results of the tests done with a dataset of 10,905 images where the selected CNN model achieved 93.12% accuracy, 93.10% macro F1 score and 98.84% ROC AUC score. It was decided beforehand to use a threshold of 0.40 while evaluating the last results of the study. A Media Pipe based face recognition prototype was created to build the system that should become an assistant for a human being instead of replacing him.

2. Introduction and Problem Background

Generative AI can create authentic looking manipulated faces. Manual supervision is hard and ineffective. Moreover, despite an opaque binary outcome, one might wrongly take it as proof of some facts. The study examines if CNN can detect some visual styles in a particular dataset and form the foundation for an image analysis assistant.

2.1 Problem statement

Develop and evaluate an AI model that classifies human face images as Real or Fake and supports a user facing image upload workflow.

2.2 Objectives

- Prepare labeled real/fake facial-image data.
- Develop and compare CNN architectures and preprocessing settings.
- Evaluate models on held out test data with class-specific and aggregate metrics.
- Select an operating threshold using validation data, not test data.
- Develop face detection and web appliction foundations.
- Document limitations, risks and planned decision support featurs.

3. Scope and AI Techniques

Scope: images of human faces only, thus excluding video, audio, identity verification and forensic proof of authenticity. The primary AI method used is supervised deep learning image classification through CNNs, while MediaPipe is utilized to carry out face detection. Also detailed in the proposal is the use of fuzzy logic for risk assessments and rule based recommendations, although the evidence does not relate how these features have already been integrated into the approach, thus they are referred to as planned features.

Earlier, the team completed some training exercises, namely the prediction of exam results based on study hours using linear regression and the classfication of images based on an apple tomato criterion through neural networks, however, they do not form a part of the final classifier.

4. Dataset and Preprocessing

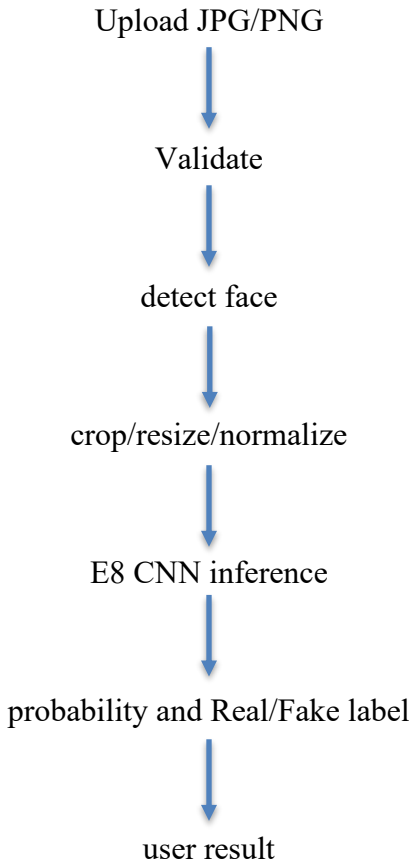
- Dataset identified in project records: Kaggle “Deepfake and Real Images” (manjilkarki/deepfake-and-real-images).
- Labels: Fake = 0, Real = 1. Dataset counts are approximately balanced.
- Image preprocessing involved loading, resizing and normalization
- The selected CNN uses 128 x 128 x 3 input.
- MediaPipe face detection was separately prototyped for uploaded images.

Split	Real	Fake	Total
Train	70,001	70,001	140,002
Validation	19,787	19,641	39,428
Test	5,413	5,492	10,905
Total	95,201	95,134	190,335

Potential methodological risks include duplicate identities across splits, provenance of manipulatd images and distribution shift. The available notes do not document an identity-disjoint split audit or independent external-dataset validation.

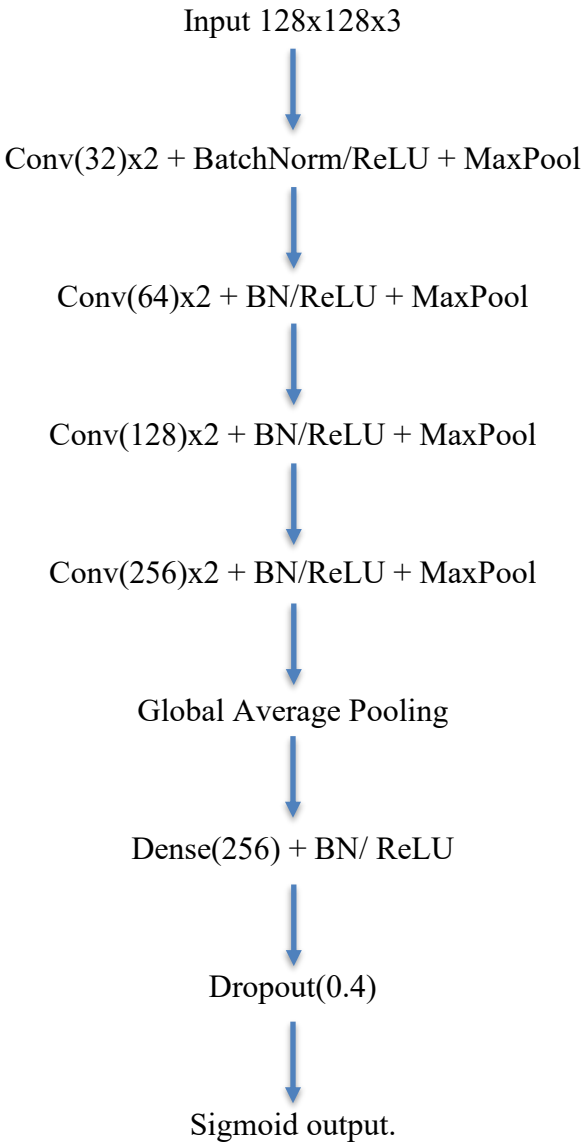
5. System Design

5.1 Intended pipeline



The frontend/backend foundation is reported as developed. The supplied project notes describe final model to web integration as remaining, so full end to end deployment is not claimd here. Proposed future layer: fuzzy risk scoring and IF THEN recommendations.

5.2 E8 architecture



For the recorded E8 scoring convention, sigmoid output represents probability of ,

Real: score $< 0.40 \rightarrow$ Fake, score $\geq 0.40 \rightarrow$ Real.

Model output scores should not be interpreted as calibrated real world probabilities without calibration anlysis.

6. Experimental Method

The trials systematically changed input resolution (128, 224, 256), augmentation, Batch Normalization, dropout, depth of the network, transfer learning (EfficientNetB0), as well as L2 regularization. E4-Ext is an extension of E4, while E5/E6 represent the BatchNorm stage present in the notes. The lastly selected one is E8, with no imputed values.

Experiment	Change	Accuracy %	Macro F1 %	ROC-AUC %
E1	Baseline 224px	88.37	88.37	95.20
E2	Augmentation	84.33	— (see note)	93.00 approx.
E3	128px baseline	89.56	89.56	96.37
E4	256px baseline	82.86	82.79	91.20
E4-Ext	Extended 256px	84.47	84.42	92.46
E5/E6	Batch normalization	88.24	88.18	96.62
E7	Dropout 0.3	88.03	87.96	96.92
E8	Deeper CNN	93.12	93.10	98.84
E9	EfficientNetB0	79.34	79.14 (derived)	87.94
E10	Dropout 0.5 + L2	92.92	92.91	98.41

Macro F1 indicates the average of the Fake and Real F1 scores, which are calculated on the basis of the confusion matrices obtained from the tests. In the records of some experiments Macros F1 for E1-E7 is given instead of the Real F1 (for instance, 88.25% for E1/E2 and 87.31% for E5/E6), but this value has been corrected. The record of the E2 states 83.06% for the macro F1 and is probably related to Real F1 as well because all of the previous records were mistakenly treated as Macro F1 while there is no confusion matrix for E2, which could clarify the case. The macro F1 for E9 is based on the confusion matrix in Figure 15. The comparison consists of the confgurations reported in the research and does not cover the complete hyperparameter runs. The baseline for 128 pixels showed better results than the baselines for 224 pixels and 256 pixels. The augmentation for E2 performed poorly in comparison with E1. The metric for EfficientNetB0 for E9 appeared to be worse than for E8 but does not imply that transfer learning methodology is not successful. .

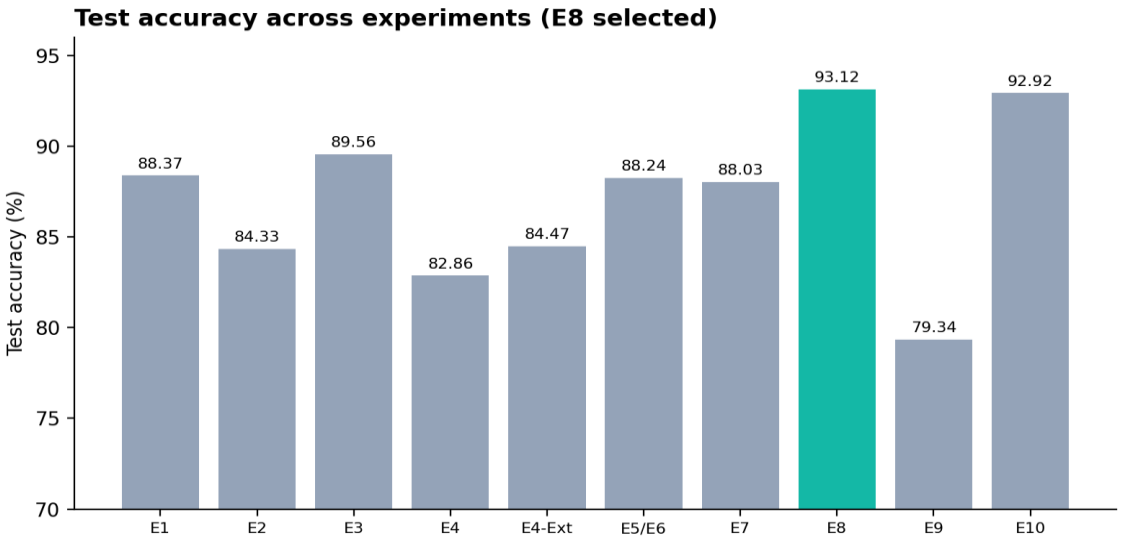


Figure 1. Test accuracy of experiments E1- E10 (E8 highlighted)

6.1 Training configuration and model summary

The experiments shared one training setup, summarised below. Individual experiments varied the settings named in their own sections (resolution, augmentation, dropout, depth, regularisation).

- **Data loading:** tf.keras.utils.image_dataset_from_directory with batch size 32, shuffled, images resized to 128 x 128 for the final model (224x224 for E1/E2, 256x256 for E4).
- **Normalization:** pixel values are rescaled by 1/255 by a Rescaling layer inside the model, so raw images can be passed to it without manual scaling.
- **Optimizer and loss:** Adam with learning rate 0.001 and binary cross-entropy. Metrics tracked were accuracy, AUC, precision and recall.
- **Callbacks:** EarlyStopping on validation loss (patience 5, best weights restored), ModelCheckpoint on validation accuracy and ReduceLROnPlateau on validation loss (factor 0.5, patience 3). The standard script allows up to 25 epochs.
- **Overfitting control:** Batch Normalzation after every convolution, dropout (0.4 before the classifier) and early stopping. No data augmentation was used in E8. Augmentation was tested only in E2.
- **Decision threshold:** 0.40, chosen on validation data because it gave the highest macro F1, rather than the default 0.50.
- **Inference preprocessing:** MediaPipe face detection crops the face with 20% padding on the bounding box so features such as hair and chin are kept.

6.2 Training code

The listings below are the standard code used across the experiments for data loading, compilation and training. Resolution and regularisation settings were changed per experiment as described above.

Listing 1. Data loading

```
import tensorflow as tf

# Standard parameters
BATCH_SIZE = 32
IMG_SIZE = (128, 128)

train_dataset = tf.keras.utils.image_dataset_from_directory(
    "data_cropped/Train",
    shuffle=True,
    batch_size=BATCH_SIZE,
    image_size=IMG_SIZE
)

val_dataset = tf.keras.utils.image_dataset_from_directory(
    "data_cropped/Validation",
    shuffle=True,
    batch_size=BATCH_SIZE,
    image_size=IMG_SIZE
)
```

Listing 2. Model compilation

```
# Standard compilation used across the CNN models
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss=tf.keras.losses.BinaryCrossentropy(),
    metrics=[
        'accuracy',
        tf.keras.metrics.AUC(name='auc'),
        tf.keras.metrics.Precision(name='precision'),
        tf.keras.metrics.Recall(name='recall')
    ]
)
```

Listing 3. Callbacks and training

```
# Callbacks to prevent overfitting and save the best model
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=5,
        restore_best_weights=True
    ),
    tf.keras.callbacks.ModelCheckpoint(
        filepath='best_model.keras',
        monitor='val_accuracy',
        save_best_only=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=3
    )
]

# Training execution
history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=25,
    callbacks=callbacks
)
```

6.3 Example notebook outputs

The following screenshots from the Google Colab notebook illustrate the model summary and evaluation printout produced for the CNN experiments. They show a 128x128 Batch Normalzation + dropout CNN with 457,409 parameters (455,937 trainable, 1,472 non-trainable), compiled with Adam (learning rate 0.001) and binary cross-entropy.

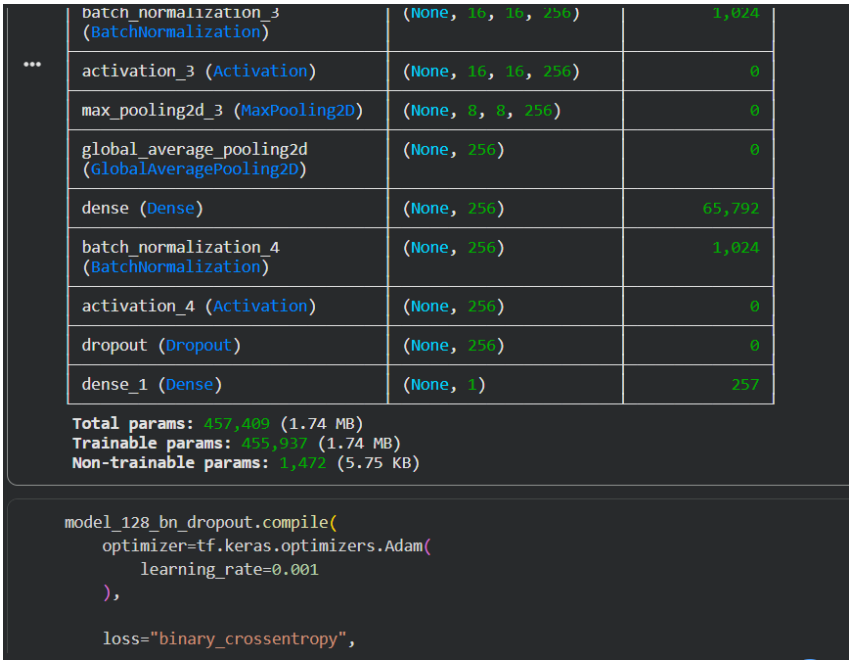


Figure 2. Keras model summary and compile cell in the Colab notebook.

Table: parameter breakdown of the model in Figure 2 (Conv and Batch Normalization parameters combined per block).

Block	Layers	Output shape	Parameters
Input	Rescaling (1/255)	128x128x3	0
1	Conv 32 + BN + ReLU, MaxPool	64x64x32	1,024
2	Conv 64 + BN + ReLU, MaxPool	32x32x64	18,752
3	Conv 128 + BN + ReLU, MaxPool	16x16x128	74,368
4	Conv 256 + BN + ReLU, MaxPool	8x8x256	296,192
Head	Global average pooling	256	0
Head	Dense 256 + BN + ReLU, Dropout	256	66,816
Output	Dense 1 (sigmoid)	1	257
Total			457,409

The evaluation cell prints accuracy, class-specific precision/recall, ROC-AUC and the confusion matrix on the 10,905-image test set (Fake = negative, Real = positive). The notebook labels this printout "Experiment 7". The decision threshold used is not shown.

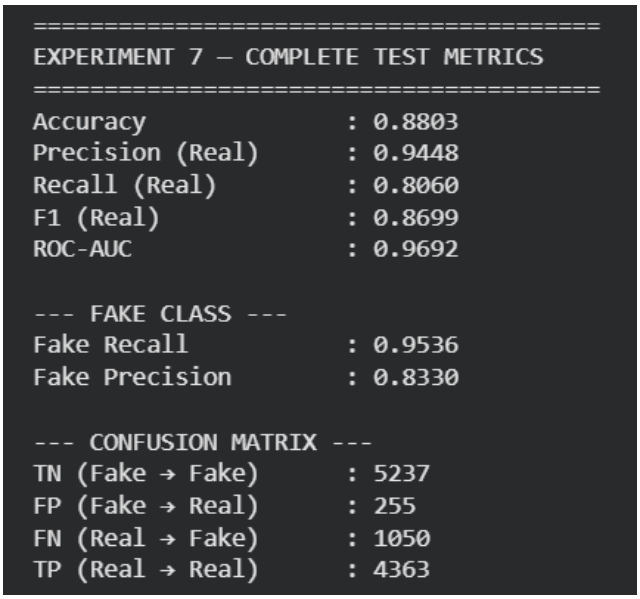


Figure 3. Complete test-metric printout with confusion matrix in the Colab notebook.

Metric (test set, 10,905 images)	Value
Accuracy	0.8803
Precision (Real)	0.9448
Recall (Real)	0.8060
F1 (Real)	0.8699
ROC-AUC	0.9692
Fake recall	0.9536
Fake precision	0.8330

Table: confusion matrix from Figure 1b (rows = actual, columns = predicted).

Actual / Predicted	Fake	Real
Fake	5,237 (TN)	255 (FP)
Real	1,050 (FN)	4,363 (TP)

Table: exported model summary of the deeper BatchNorm CNN (model name "mediapipe_e08_deeper_batchnorm", 1,243,169 parameters in total, of which 1,240,737 are trainable and 2,432 non-trainable).

Block	Layers	Output shape	Parameters
Input	Input layer, Rescaling (no augmentation)	128x128x3	0
1	Conv 32 + BN, Conv 32 + BN, MaxPool, Dropout	64x64x32	10,400
2	Conv 64 + BN, Conv 64 + BN, MaxPool, Dropout	32x32x64	55,936
3	Conv 128 + BN, Conv 128 + BN, MaxPool, Dropout	16x16x128	222,464
4	Conv 256 + BN, Conv 256 + BN, MaxPool, Dropout	8x8x256	887,296
Head	Global average pooling	256	0
Head	Dense 256, BatchNorm, Dropout	256	66,816
Output	Dense 1 (sigmoid)	1	257
Total			**1,243,169**

6.4 Training time and computational cost

Training was costly. The logged Google Colab run (Tesla T4 GPU) processd 4,376 steps per epoch in association with 140,002 training images and a batch size of 32. Epochs 6–16 took 542–563 seconds (average around 549 s or 9.2 min). The screenshot captured after the next two hours and thirty-three minutes of the session at the epoch 17 of 30. The mean this session indicated suggests an entire duration of about 4.6 hours for all thirty epochs (not considering early stopping). The metrics shown in the log are those derived from the training set, but the log does not explicitly state what experiment was conducted. The epoch-14 training accuracy measured during the logging was about 96.9%, which makes this run different from the E8 run (epoch-14 training accuracy of approximately 99.8% is illustrated in Figure 13). The documentation indicates different planned epochs (30 according to this log and 25 in the ordinary script).

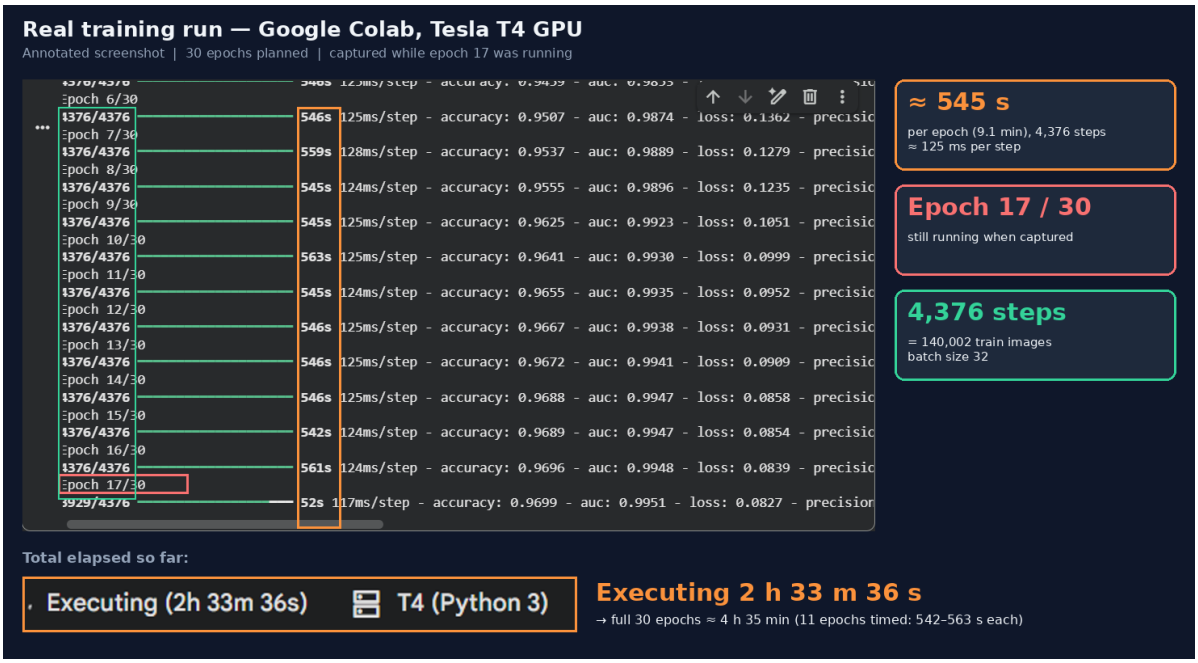


Figure 4. Annotated training log from a Colab run (Tesla T4) showing epoch progress, time per epoch and elapsed execution time. This log is not the E8 training history (see Figure 15).

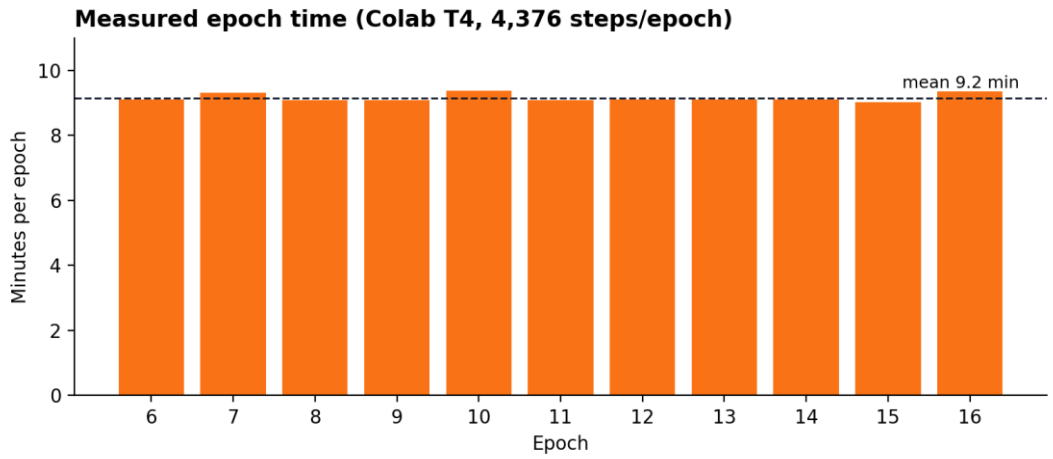


Figure 5. Measured time per epoch (epochs 6–16).

This cost explains why the experiment series was run as a limited set of configurations rather than an exhaustive hyperparameter search and why checkpoint-based early stopping was valuable.

7. Threshold Selection and Model Evaluation

Validation results were employed in the selection of the E8 threshold, after which it was fixed prior to test evaluation. The validation set comprised 39,428 images. At threshold of 0.40, validation accuracy and macro F1 were at 98.05%. The validation ROC-AUC was 99.43%. The discrepancy from the held-out test accuracy (93.12%) warrants an investigation of potential sources of optimism including possible distribution differences or leaks.

Validation results were employed in the selection of the E8 threshold, after which it was fixed prior to test evaluation. The validation set comprised 39,428 images. At threshold of 0.40, validation accuracy and macro F1 were at 98.05%. The validation ROC-AUC was 99.43%. The discrepancy from the held-out test accuracy (93.12%) warrants an investigation of potential sources of optimism including possible distribution differences or leaks.

The number 93.12% indicates the portion of test images that have been classified by the model correctly. However, this does not imply that any particular image has been captured correctly, for it reflects the model's confidence score but not the probability of it being true.

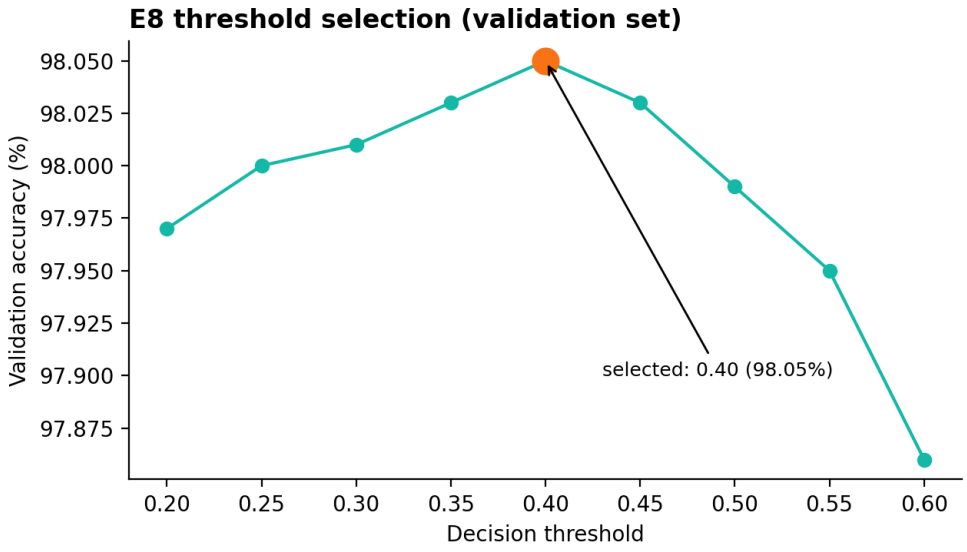


Figure 6. Validation accuracy versus decision threshold for E8, 0.40 was selected before test evaluation.

E8 held-out test metric	Value
Test samples	10,905
Accuracy	93.12%
Macro precision	93.47%
Macro recall	93.09%
Macro F1	93.10%
ROC-AUC	98.84%
Fake precision / recall / F1	89.74% / 97.49% / 93.45%
Real precision / recall / F1	97.21% / 88.69% / 92.76%
Locked decision threshold	0.40

7.1 Confusion matrix

- Rows = actual class
- columns = predicted class.
- The source uses Fake = negative (0), Real = positive (1).

Actual / Predicted	Fake	Real
Fake	5,354	138
Real	612	4,801

In total, there were 138 fake images that were wrongly classified as real, as well as 612 real images that were wrongly classified as fake. There are implications of mistakes in either direction: missing a fake will result in political misinformation, while falsely accusing someone of malpractice can damage trust or reputation.

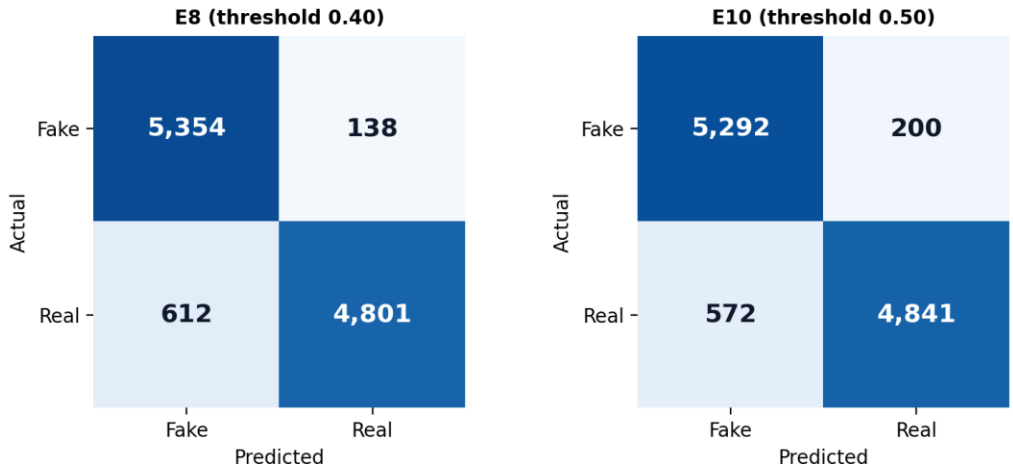


Figure 7. Confusion matrices for E8 and E10 on the 10,905-image test set (rows = actual, columns = predicted).

7.2 E8 versus E10

E10 used dropout 0.5 as well as L2=1x10^{-4} and resulted in 92.92% test accuracy, 92.91% macro F1 score and 98.41% ROC AUC. E10 brought about real recall to 89.43%, which is higher than E8's 88.69%, but the fake recall decreased from 97.49% to 96.36%. E8 gets credit for the best overall test performance, it is worth noting that E8's evaluation took place with a validation threshold of 0.40, therefore not making for a proper like-for-like comparison with E10 being evaluated at 0.50.

8. Implementation Status and Testing

Component	Evidence-based status
Linear regression practice	Completed practice model
Apple/tomato DL practice	Completed practice classifier
MediaPipe face detection	Prototype developed
CNN deepfake experiments	E1–E10 evaluated
E8 classifier	Selected on validation performance, final test metrics reported
Web frontend/backend	Foundations developed, final inference integration unverified
Fuzzy logic / expert rules	Proposed, completed integration not evidenced
Production deployment	Not established by supplied records

9. Ethical and Social Considerations

This particular system requires that sensitive information, in the form of images of people's faces, is being handled properly. It is very important that consent has been provided, images should be retained for the necessary time and uploaded images should be secured properly. Also, the compositions of various data sets may affect the performance in the case of different skin colors and shades, age groups, qualities of the images and types of the generators used. Overall, predictions should serve only as indications rather than proof and the presentation of such predictions should be accompnied with adequate disclaimers concerning the possibility of obtaining false results.

10. Limitations and Future Work

The detector is image-only and dataset-dependent. The accuracy of 93.12% that is given in the discussion is not applicable to any further synthetic images generated in the future. The Real→Fake errors are significant and no cross-dataset/generalzation tests are available. Moreover, it is recommended to conduct additional research on issues such as identity-disjoint split verification, the impact of external generators, score calibration, explainability of algorithms (such as Grad-CAM), multiple-face processing, web integration and fuzzy-logic risk scoring.

11. Conclusion

DeepGuard AI shows a thorough progressive CNN building technique for differentiating fake from real images of faces. The experiments disclosed that a deeper batch-normalized CNN uses 128x128 inputs is the most effective one according to the best reported results on held-out testing. The project also laid the practical base for computer vision and software. Proper implementation needs further integration, outside validation and good communication concerning uncertainties.

12. Detailed Theory and Technical Background

12.1 Supervised learning and classification

Supervised learning applies known examples to forecast the relationship between inputs and outputs. Different from the previous study-hours-to-marks regression activity, the goal here is to carry out a binary image classfication, where the model predicts whether a face is a member of the Real (1) or Fake (0) class. While training attempts to reduce a classification loss, validation assesses the quality of prediction based on the instances not used in weight updating.

12.2 Convolution, activation and pooling

With the aid of small filters, a convolutional layer can scan areas of the image and generate feature maps. For example, filters in early blocks can react to edges and color changes while consecutive blocks combine the outcomes of filters for richer outputs. Rectified Linear Unit (ReLU) activation makes a non-linear model. Pooling reduces the size of the model while keeping strong activations. E8 design prefers using the stack of blocks instead of a simple classifier.

12.3 Batch normalization and regularization

Batch normalization applies batch statistics to normalize activations when training and computes its running statistics while inferring, thus providing stable optimization. Dropout randomly removes some activations in the training phase. E8 applied a dropout of 0.4. E10 tried a stronger dropout (0.5) plus L2 regularization of weights ($1x10^{-4}$). These modifications are not guaranteed to increase effectveness, therefore they were assessed experimentally.

12.4 Model scores and decision thresholds

The sigmoid layer generates a numerical score linked to the Real class. By means of a threshold, the given score can turn into a class label. E8 used the validation-selected threshold value of 0.40: all values lower than this threshold are coded Fake, while those equal to or higher than the threshold are coded Real. Neural score does not automatically represent a calibratd probability of authenticity, as the initiative failed to establish calibration in real image sources

12.5 Computer vision and the three-concept requirement

The final work is expected to demonstrate supervised deep learning classification and computer vision face detection/preprocesing. The proposal also included fuzzy logic risk assessment and rule-based reasoning. Given this, it is possible to conclude that the experimental record that is presented does not prove the implementation of these last modules and their design is justified only. The team should provide such explanation in the reports.

13. Dataset Methodology and Reproducibility

13.1 Source, class mapping and splits

The Kaggle Deepfake and Real Images dataset generally comprises a training dataset with 140,002 images, a validation dataset with 39,428 images and a test dataset with 10,905 images. There are a total of 95,201 Real images and 95,134 Fake images across the entire dataset, resulting in a total of 190,335 images. The training split is perfectly balanced, while the validation and test splits are approximately balanced.

13.2 Data-processing steps

- The images in directories labeled with their respective classes have to be read and the split membership preserved.
- The first thing to do is to check if the images could be decoded and check that label mappings used are Fake=0 and Real=1.
- Change images' resolution to fit that of the experiment that is to be run (128x128 for E8).
- Pixel normalization has to be performed in a uniform way across training, validation, testing and inference.
- Perform augmentation processes only during training steps of augmenting.
- The training data have to be used to give weights to the model, validation data will be used to help with checking points and extension selection.
- Make evaluations on the model configuration on held-out test splits.

13.3 Quality controls and unresolved checks

No evidence has been provided regarding the efficacy of tested or disjoint identity datasets or the overlapping dataset spanning. All these verifications matter because what is done in the validation can impose wrong asumptions and inflate estimates if data points or identities across subsets are concatenated. In addition to checking if some splits were conducted in similar conditions, it is important to analyze the extent to which the validation step is indicative of the testing outcome.

14. Expanded Experiment-by-Experiment Discussion

14.1 Experimental progression and rationale

The progression of the ten experiments was methodical. In the first four experiments (E1-E4), the input resolution was varied and data augmentation was performed to arrive at a suitable size. In E5-E7 Batch Normalzation and dropout were introduced to stabilize the training process and overcome overfitting at the determined resolution size of 128 x 128. In E8 the architecture of the network was deepened – this led to the greatest improvements. Finally, E9 and E10 tested transfer-learning methods and stronger regularization in comparison to E8. The table shows the summary of purposes and changes of each of the experiments, while the further sections present detailed results.

Exp.	Goal	What changed and why	Test accuracy
E1	Establish a benchmark	Basic CNN at 224x224 with no augmentation, Batch Normalization or dropout, to give a baseline for later comparison.	88.37%
E2	Test whether augmentation improves generalization	E1 plus data augmentation, intended to stop the model memorizing specific faces. The specific transforms are not documented here.	84.33% (lower)
E3	Reduce training cost, test whether lower resolution keeps enough detail	Same baseline CNN at 128x128. Inputs are tightly cropped faces, so less resolution may be needed.	89.56% (better than E1)
E4	Test whether higher resolution exposes finer artifacts	Same baseline CNN at 256x256, at much higher computational cost. E4-Ext trained the same model for longer.	82.86%, E4-Ext 84.47%
E5/E6	Stabilize learning	Batch Normalization added at 128x128. E6 is recorded as a variant of E5 and has identical recorded results.	88.24%

E7	Reduce overfitting	Dropout 0.3 added to the Batch Normalization CNN, so the model cannot rely on a few memorized features.	88.03% (ROC-AUC 96.92%)
E8	Increase model capacity	Deeper CNN: two convolution layers per block (earlier models used one) across four blocks of 32–256 filters, with Batch Normalization, global average pooling, Dense(256) and dropout 0.4. No data augmentation.	93.12% (selected)
E9	Test transfer learning against custom CNNs	Pre-trained EfficientNetB0 at 128x128 instead of a custom network.	79.34%
E10	Test stronger regularization	E8 architecture with dropout raised to 0.5 and L2 weight regularization ($1x10^{-4}$).	92.92%

14.2 E1 - 224x224 baseline

This reference CNN achieved 88.37% test accuracy, 88.37% macro F1 and 95.20% ROC-AUC. It established a starting point for later comparisons.

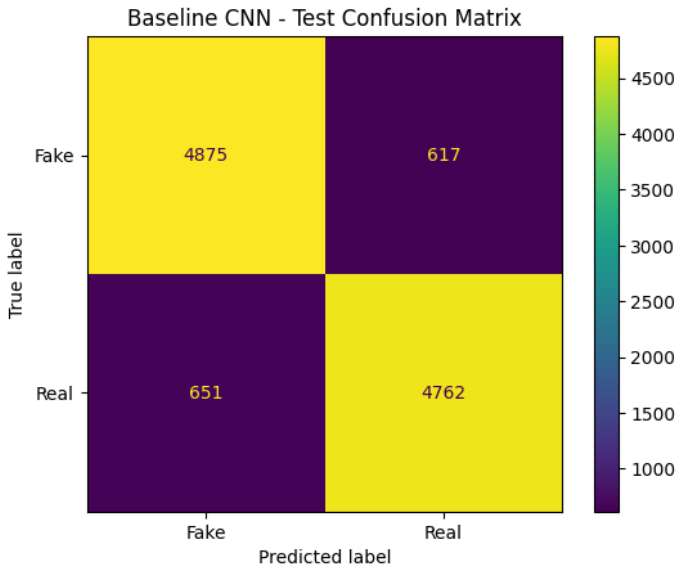


Figure 8. E1 baseline CNN (224x224) test confusion matrix.

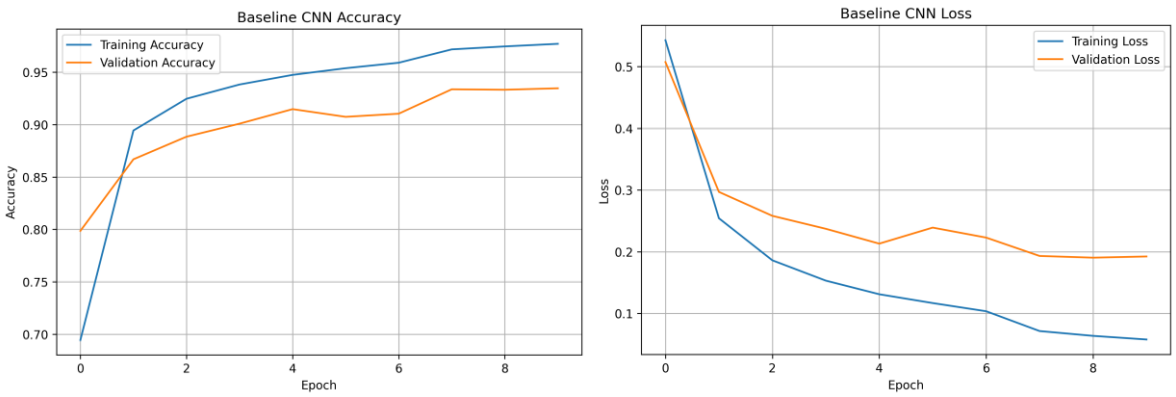


Figure 9. E1 baseline CNN: training/validation accuracy (left) and loss (right), epoch axis starts at 0.

14.3 E2 - augmentation

Using the augmentation setup tested in this analysis dropped the test accuracy to 84.33% (the F1 of 83.06% noted here is likely the F1 of the Real class, refer to the footnote at the bottom of this table for more information). However, this does not imply that augmentation is intrinsically bad. It means that the confguration of transformations and parameter settings used did not provide any added value to this configuration.

14.4 E3 - reduced input resolution

At 128x128, the baseline reached 89.56% of accuracy, 89.56% of macro F1 and 96.37% of ROC-AUC. It outperformd the larger 224x224 baseline.

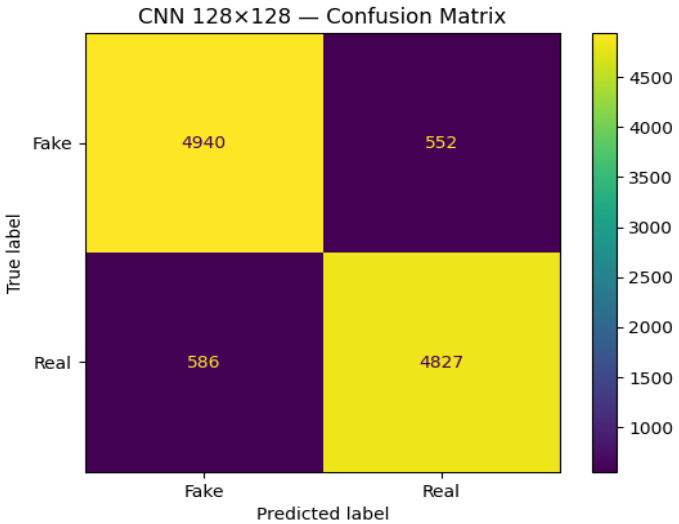


Figure 10. E3 CNN (128x128) test confusion matrix.

14.5 E4 and E4-Ext - higher resolution

The 256*256 model demonstated an accuracy of 82.86%, while longer training brought the model to 84.47%, still under the performance of E3.

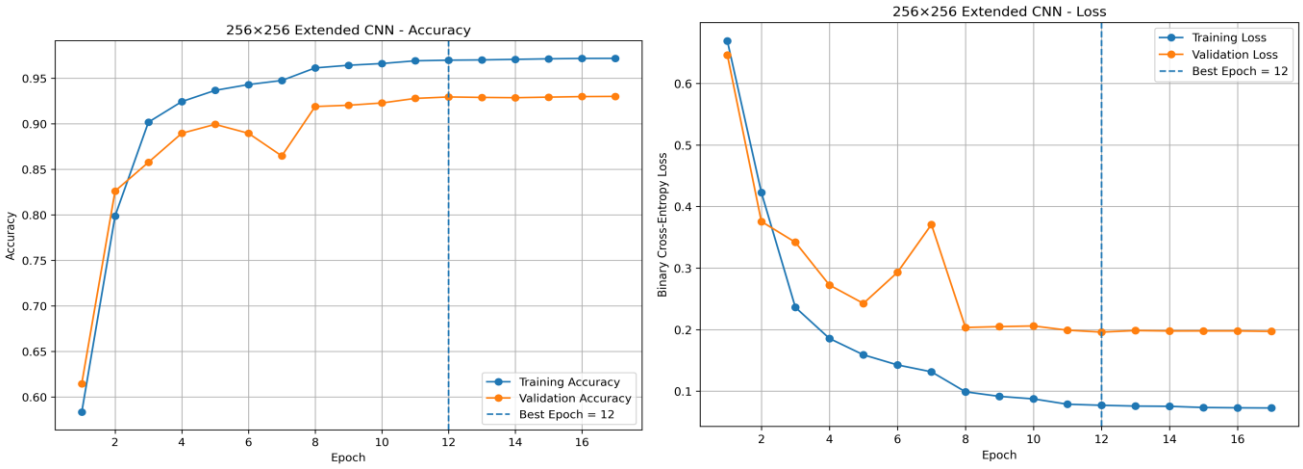


Figure 11. E4-Ext (256x256 extended CNN): training/validation accuracy (left) and loss (right), best epoch = 12.

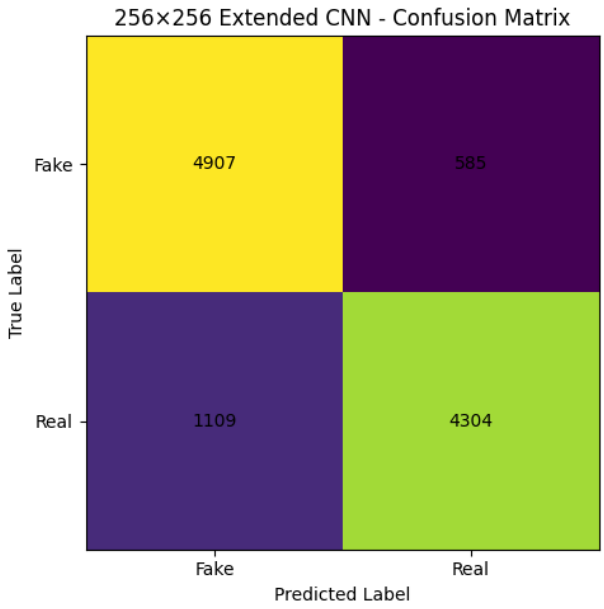


Figure 12. E4-Ext (256x256 extended CNN) test confusion matrix.

14.6 E5/E6 - batch-normalization stage

At this stage, the accuracy of the model was prolonged by 88.24% ROC AUC. There were improvements in some parameters, but it did not allow the model to surpass experiment E3 as far as the acuracy is concerned.

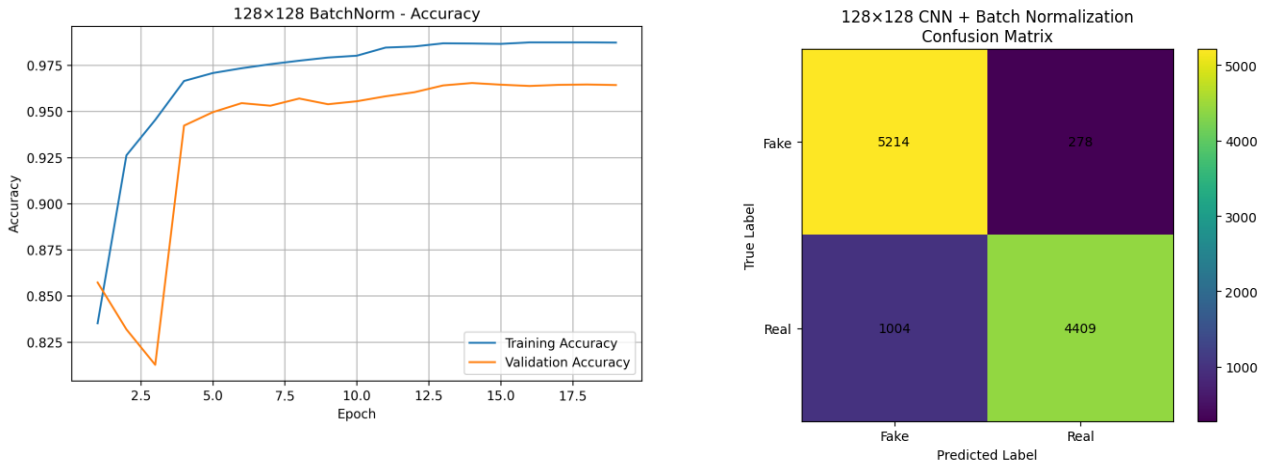


Figure 13. 128x128 BatchNorm stage (E5/E6, the exported E5 and E6 graphs are identical): training/validation accuracy (left) and test confusion matrix (right).

14.7 E7 - dropout 0.3

The clasifier showed an accuracy of 88.03% and ROC AUC of 96.92%. The improvment in the ranking capacity did not lead to any increase in the accuracy at the threshold obtained.

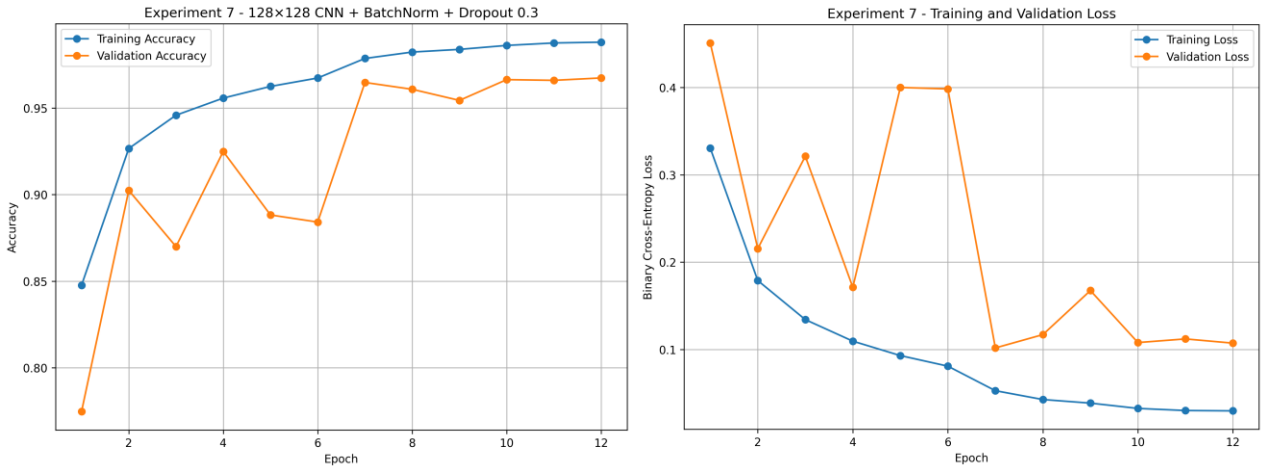


Figure 14. E7 (BatchNorm + dropout 0.3): training/validation accuracy (left) and loss (right).

14.8 E8 - deeper CNN with dropout 0.4

The more complex model of BatchNorm CNN of resolution equal 128*128 showed 93.12% of accuracy, 93.10% of macroF1 and 98.84% of ROC AUC and was used for classification.

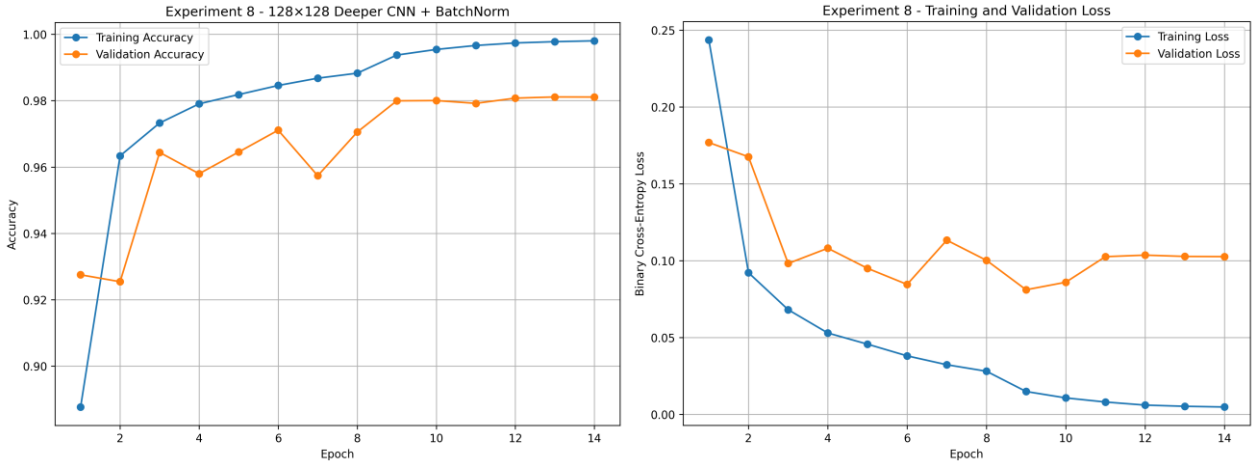


Figure 15. E8 (deeper BatchNorm CNN): training/validation accuracy (left) and loss (right), epochs 1–14 as plotted.

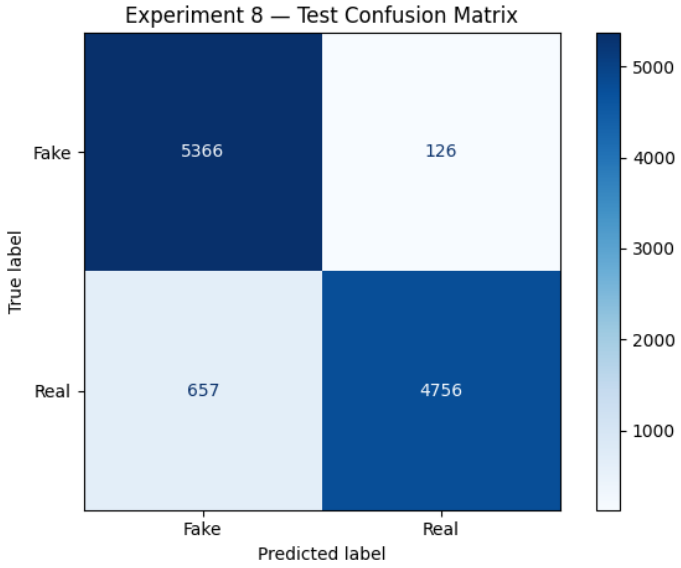


Figure 16. E8 test confusion matrix as exported from the notebook (5,366 / 126 / 657 / 4,756). The locked-threshold (0.40) matrix used for the reported final results is 5,354 / 138 / 612 / 4,801

14.9 E9 - EfficientNetB0

The transfer learning model used in the tests demonstrated an accuracy of 79.34% and ROC AUC of 87.94%. It should be mentiond that the assesment is limited only to the application of the chosen configuration.

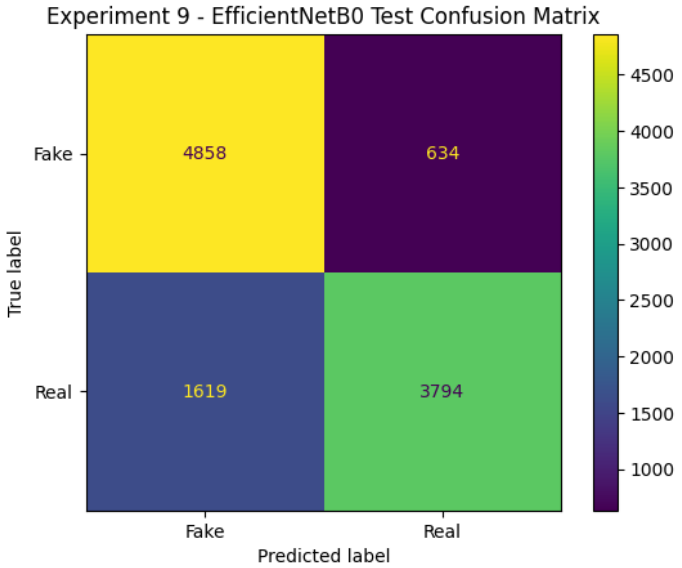


Figure 17. E9 (EfficientNetB0) test confusion matrix.

14.10 E10 - additional regularization

The utilization of dropout 0.5 in combination with L2 Regularzation have yielded an accuracy of 92.92% and an ROC AUC of 98.41%. The real recall parameter has increased marginally, but the remaining parameters F1 and AUC scores are still well below the results obtaind in experiment E8.

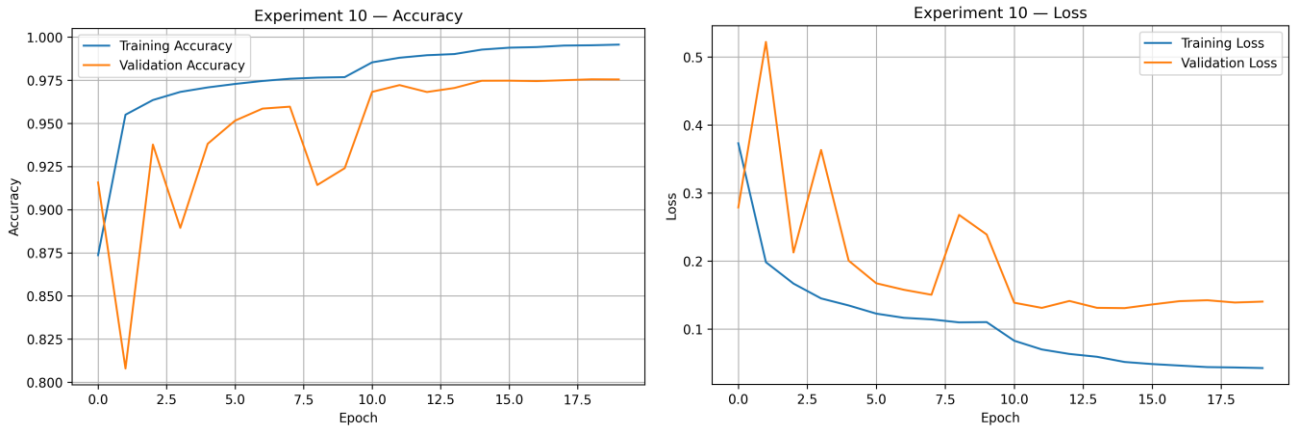


Figure 18. E10 (dropout 0.5 + L2): training/validation accuracy (left) and loss (right), epoch axis starts at 0.

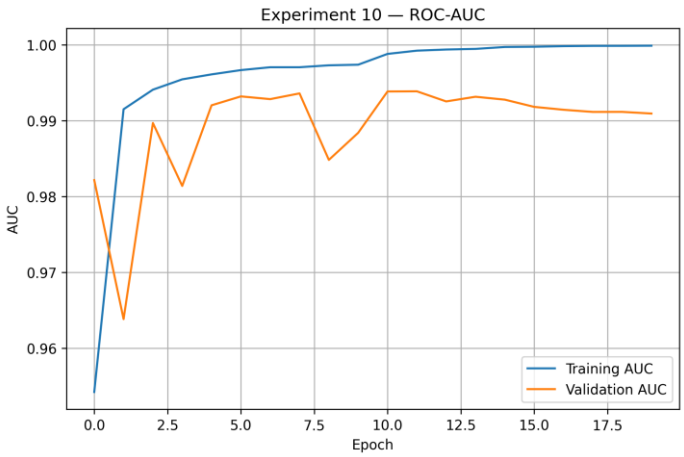


Figure 19. E10 training and validation ROC-AUC per epoch.

15. Quantitative Evaluation and Error Analysis

15.1 Metric definitions

Accuracy indicates the ratio of all classified samples that are accurate. Precision for some class gives information regarding the correctness of predictions about that class, while recall indicates the proportion of the members of the class that were recovered. The F1 measure is consequntially calculated as the average of precision and recall. The macro average would consider both classes equally. The ROC-AUC number indicates the ranking discrimnation at a range of thresholds.

15.2 Detailed E8 evaluation

Measure	Validation	Held-out test
Number of images	39,428	10,905
Selected threshold	0.40	0.40 (locked)
Accuracy	98.05%	93.12%
Macro F1	98.05%	93.10%
ROC-AUC	99.43%	98.84%
Fake recall	97.91%	97.49%
Real recall	98.18%	88.69%

According to the confusion matrix, there are 5,354 correctly classified Fake images, as well as 138 Fake images incorrectly classified as Real, 612 Real images incorrectly classified as Fake and 4,801 correctly clasified Real images. This indicates that some altered images are not accurately identified, while some genuine images have been incorrectly labeld. Both types of error need to be disclosed in any application that is meant for public verification.

15.3 E8 versus E10 trade-off

Metric	E8	E10
Accuracy	93.12%	92.92%
Macro F1	93.10%	92.91%
ROC-AUC	98.84%	98.41%
Fake recall	97.49%	96.36%
Real recall	88.69%	89.43%
Fake incorrectly labeled Real	138	200
Real incorrectly labeled Fake	612	572

E10 had a decrease in its true positive flags, with the flagging from 612 for actual images reduced down to 572. Unfortunatly, the false negatives went up, with an increase from 138 to 200. The adopted E8 shows the better overall figures as reported. Any real application would have to reconsider thresholds based on the severity of each error.

15.4 Interpretation of the validation/test gap

The validation accuracy was recorded at 98.05% in E8 while held-out test accuracy was 93.12%, thus giving a difference of 4.93 percentage points. Reasons for the deviations can be many, including distribution differences, model-selection optimism and sampling attributes. The current record does not determine which of the mentiond reasons is applicable. Further independent validation is needed before making any statement regarding general reliability in real life.

15.5 Corrected experiment summary

The correlation values, ROC-AUC and thresholds have been taken from the experimentation record, while macro precision, macro recall and macro F1 have been calculated from the confusion matrices in Section 15.6. The E9 values are formed from the confusion matrix above, while a dash indicates the value is not available in the records and its confusion matrix cannot be determind.

Exp.	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC	Test threshold
E1	88.37%	88.37%	88.37%	88.37%	95.20%	0.50
E2	84.33%	—	—	—	~93.00%	0.50
E3	89.56%	89.57%	89.56%	89.56%	96.37%	0.50
E4	82.86%	83.34%	82.82%	82.79%	91.20%	0.50
E4-Ext	84.47%	84.80%	84.43%	84.42%	92.46%	0.50
E5/E6	88.24%	88.96%	88.20%	88.18%	96.62%	0.50
E7	88.03%	88.89%	87.98%	87.96%	96.92%	0.50
E8	93.12%	93.47%	93.09%	93.10%	98.84%	0.40

E9	79.34%	80.34%	79.27%	79.14%	87.94%	0.50
E10	92.92%	93.14%	92.90%	92.91%	98.41%	0.50

15.6 Test confusion matrices

- Rows = actual class
- columns = predicted class
- Fake = negative (0)
- Real = positive (1)
- Test set = 10,905 images

Exp.	TN (Fake→Fake)	FP (Fake→Real)	FN (Real→Fake)	TP (Real→Real)	Total
E1	4,875	617	651	4,762	10,905
E2	—	—	—	—	10,905
E3	4,940	552	586	4,827	10,905
E4	4,880	612	1,257	4,156	10,905
E4-Ext	4,907	585	1,109	4,304	10,905
E5/E6	5,214	278	1,004	4,409	10,905
E7	5,237	255	1,050	4,363	10,905
E8	5,354	138	612	4,801	10,905
E9	4,858	634	1,619	3,794	10,905
E10	5,292	200	572	4,841	10,905

15.7 Class-by-class test results

- Fake precision = TN/(TN+FN)
- Fake recall = TN/(TN+FP)
- Real precision = TP/(TP+FP)
- Real recall = TP/(TP+FN).
- All values are computed from the confusion matrices above.

Exp.	Fake precision	Fake recall	Fake F1	Real precision	Real recall	Real F1
E1	88.22%	88.77%	88.49%	88.53%	87.97%	88.25%
E2	—	—	—	—	—	—
E3	89.40%	89.95%	89.67%	89.74%	89.17%	89.46%
E4	79.52%	88.86%	83.93%	87.16%	76.78%	81.64%
E4-Ext	81.57%	89.35%	85.28%	88.03%	79.51%	83.56%
E5/E6	83.85%	94.94%	89.05%	94.07%	81.45%	87.31%
E7	83.30%	95.36%	88.92%	94.48%	80.60%	86.99%
E8	89.74%	97.49%	93.45%	97.21%	88.69%	92.76%
E9	75.00%	88.46%	81.18%	85.68%	70.09%	77.11%
E10	90.25%	96.36%	93.20%	96.03%	89.43%	92.62%

15.8 Training and validation results

Best epoch is the checkpoint taken by early stopping. E8 completed its 14 epochs, whereas E10 completed its 20 epochs and managed to reach the best checkpoint at the 15th epoch.

Exp.	Epochs run	Best epoch	Best val. accuracy	Best val. AUC	Best val. loss
E1	10	9	—	—	—
E3	10	8	93.36%	0.9823	0.17525
E4	10	10	92.71%	0.9776	0.2095
E4-Ext	17	12	92.95%	0.9796	0.1960
E5/E6	19	14	96.54%	0.9912	0.1145
E7	—	7	96.48%	0.9927	0.1018
E8	14	9	≈98.11%	0.9943	0.0810
E10	20	15	97.46%	0.9928	0.13068

15.9 E8 and E10 validation results

The ROC-AUC for E10 validation was equal to 99.29% (the checkpoint from epoch-15). One earlier record reportd 99.667%, although it was inconsistent with the figure in Figure 19, hence it was not considered.

Metric	E8 validation	E10 validation
Samples	39,428	39,428
Threshold	0.40	0.50
Accuracy	98.05%	97.46%
Macro precision / recall / F1	98.05% / 98.05% / 98.05%	97.47% / 97.46% / 97.46%
ROC-AUC	99.43%	99.28%
Fake precision / recall / F1	98.16% / 97.91% / 98.04%	98.00% / 96.88% / 97.44%
Real precision / recall / F1	97.93% / 98.18% / 98.06%	96.94% / 98.04% / 97.49%
TN / FP / FN / TP	19,231 / 410 / 360 / 19,427	19,029 / 612 / 388 / 19,399

15.10 Experiment hyperparameter summary

Exp.	Resolution	Batch Norm	Dropout	L2	Depth	Main purpose
E1	224x224	No	—	—	Baseline	Establish baseline
E2	224x224	No	—	—	Baseline	Test augmentation
E3	128x128	No	—	—	Baseline	Test lower resolution
E4	256x256	No	—	—	Baseline	Test higher resolution
E4-Ext	256x256	No	—	—	Extended training	Test additional training
E5/E6	128x128	Yes	—	—	CNN	Test Batch Norm
E7	128x128	Yes	0.30	—	CNN	Test dropout
E8	128x128	Yes	0.40	—	Deeper CNN	Increase feature capacity
E9	128x128	Built-in	—	Built-in	EfficientNetB0	Test transfer learning
E10	128x128	Yes	0.50	$1x10^{-4}$	Same as E8	Stronger regularization

16. Designed Risk Assessment and Rule-Based Advice

As per the initial proposal, a fuzzy risk module will be utilized to evaluate model confidence and image quality parameters including but not limited to resolution, compression and illumination stability. The fuzzy risk module would inform about risk level as either Low, Medium or High. Elements that will provide the fuzzy risk module input are Low/Medium/High confidence level, Poor/Acceptable/Good image quality and Low/Medium/High manipulation risk. It is also noted that.

Example conceptual condition	Illustrative advice (not tested output)
No face detected	Request a clear image containing a face
High manipulation risk	Request independent verification before sharing
Medium or uncertain risk	Flag for manual review
Low risk	Likely authentic under this model, not proof of authenticity

Future experiments need to assess the rules based on relevant tests, explain how uncertanty and quality of image affect each other and refrain from getting the final result from the uncalibrated model score.

17. Software Architecture and Prototype Verification

It was stated that there is user-friendly UI, a platform for uploading, analyzing and clasifying pictures and displaying the outcomes. The following notes proposed the description of frontend/backend, but no evidence of succesful inference was provided. Therefore, it is defined as a system of architecture but not as a functional system.

- Frontend: allowing selection of JPG/PNG, displaying preview and indicating prediction and limitations.
- Backend: checking the image format and size, receiving the image securely and running preprocessing and model inference.
- Face processing: MediaPipe prototype is used to recognize face areas, while production of multiple faces and errors is subject to testing.
- Model service: E8 weights must be loaded and the same preprocesing procedure 128x128 and threshold of 0.40 must be applied.
- Output: Real/Fake label, score, alert about uncertainty and optional future risks.
- Data management: ensure that facial images are not stored unnecessarily, obtain consent and ensure the storage of images is documentd.

18. Testing Plan, Risks and Ethical Considerations

Test scenario	Expected behavior	Status from supplied record
Clear image with one face	Accept image, generate classification	Classifier tested on held out data, app integration unverified
Image with no face	Show validation/face detection error	MediaPipe prototype exists, end-to-end unverified
Corrupt or unsupported upload	Reject safely	Not evidenced
Low resolution or compressed face	Warn about reduced reliability	Not evidenced
Multiple faces	Apply documented handling rule	Not evidenced
New generator / external dataset	Measure performance shift	Not evidenced

Ethical risks include issues such as privacy and consent when procesing facial images, bias related to the data set representation, false accusations of altered images and the risk of appearing to trust fakes that have not yet been realized. In order to handle such an application ethically, secure upload proceses should be put into place, the amount of data collected should be kept to a minimum, warnings should be made known and indpendent human verification should be made mandatory for significant decisions.

19. Project Management and Four-Member Participation

The initial proposal identifies four contributors and allocates specific roles for each. The final party must adapt the following table so that it corresponds to the actual practice. The following are original responsiblities.

Member	Index	Proposal responsibility	Final evidence to attach
K.D.S.T. Nethmi	D/DBA/25/0039	CNN training and evaluation	Experiment notebooks, saved E8 model, metrics
S.P.K.T.C. Perera	D/BIS/24/0007	Fuzzy logic and expert rules	Risk-rule design, implementation/tests if completed
C.S. Gallage	D/BIT/24/0078	Frontend/backend and testing	Application code, screenshots, integration tests
S.S.T. Perera	D/BCS/25/0011	Coordination, literature, data sourcing and preprocessing	Dataset notes, preprocessing scripts, documentation